

MDD: Process Drift Detection in Event Logs Integrating Multiple Perspectives

1st Yunuo Cao

Information Engineering College,
Capital Normal University
Beijing, China
Yunuo@cnu.edu.cn

2nd Leilei Lin*

School of Management,
Capital Normal University
Beijing, China
leilei_lin@cnu.edu.cn

3rd Ying Di

School of Management,
Capital Normal University
Beijing, China
yingd3357@gmail.com

4th Xiaohe Li

Aerospace Information Research Institute,
Chinese Academy of Sciences
Beijing, China
lixiaohe@aircas.ac.cn

5th Wenlong Chen

Information Engineering College,
Capital Normal University
Beijing, China
chenwenlong@cnu.edu.cn

Abstract—Process-aware information systems (PAIS) are designed to manage and support business processes within an organization. Drift detection in process mining aims to detect process changes by analyzing event logs, which can guarantee the accuracy and reliability of business processes within PAIS. However, existing methods often only consider process changes from a single perspective (e.g., control-flow), without fully utilizing other information in event logs like resources and timestamps. In this paper, we propose a novel Multi-view Drift Detection method, called MDD, to detect changes with graph convolutional networks. Specifically, 1) multiple k-nearest neighbors graphs are constructed for two log segments based on multiple perspectives, and graph convolutional networks are employed to capture features from these graphs. Meanwhile, we utilize a fully connected network to fuse multiple knn graphs, generating two fusion graphs; 2) we use Wasserstein distance to measure the distance between two fusion graphs after feature fusion; 3) the K-means algorithm is adopted to find the candidate drift points, and then the actual change points will be identified with a filter mechanism. Experimental results demonstrate that the MDD effectively identifies drift points on simulated logs and real-life logs.

Index Terms—Business process management, Process evolution, Graph fusion, Wasserstein distance

I. INTRODUCTION

The business process is a series of interconnected activities that a company executes internally to achieve specific business objectives. However, to maintain a competitive advantage in the rapidly changing market, companies actively strive to continuously optimize and improve their business processes. In this context, Business Process Management (BPM) has become an important research field, aiming to explore how to maximize business efficiency. Process mining is the bridge between business process management and data mining, which can mine valuable knowledge (e.g., resources, roles, departmental structure, etc.) from event logs for setting and improv-

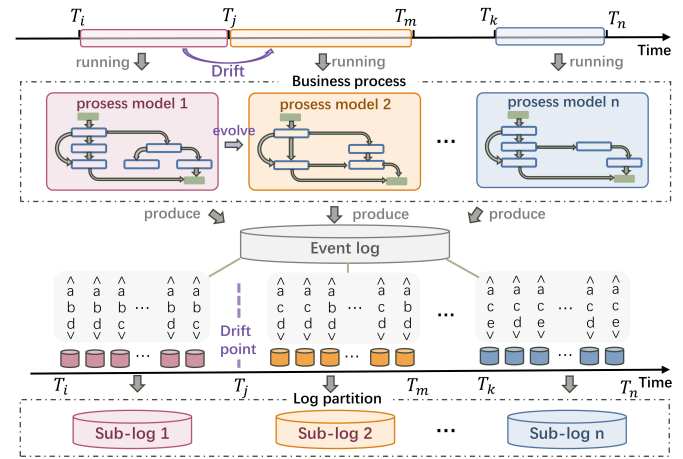


Fig. 1: An example for illustrating the concept drift in process mining

ing ongoing business processes [1]. Three classic applications of process mining in real-world scenarios, include process discovery [2] [3], conformance checking [4] and process enhancement [5]. But, these techniques usually assume that processes are in a steady state that an event log corresponds to only one process model. In real life, upgrades or changes in the requirements of an organization may lead to the adjustment of business processes, and the logs that record the business execution process may also evolve. If we directly mine the logs, we would obtain a highly complex and inaccurate process model, which is mixed with different versions of business processes.

Concept drift detection is a family of methods to find the drift points (a.k.a, change points) for partitioning the event logs into many sub-logs, which helps in obtaining a more accurate process model in process mining. As shown in Fig.1, as time passes, business processes running on an information system evolve from the T_j moment into new

Leilei Lin is the corresponding author with capital normal university.
E-mail: leilei_lin@cnu.edu.cn

process models. This change can be planned or may occur due to unforeseen circumstances, but all these changes are recorded in the database in the form of an event log. If we directly perform process discovery on the log, the resulting model will not be accurate. Therefore, it is necessary to employ drift detection methods to identify the drift points (e.g., T_j , T_m , T_k) in the log and subsequently partition them into sub-logs. It is worth emphasizing that each trace represents a moment in time.

As we know, many excellent drift detection algorithms have been proposed in the past decade [5-11]. However, they still have several limitations, which can be summarized as follows: 1) Only focusing on a single perspective. Bose et al. [6] [7] used hypothesis testing methods to analyze sudden drift and gradual drift in event logs, both of which rely solely on the control flow perspective for detection. 2) Low accuracy. Barbon et al. [8] used density clustering to consider both the activity perspective and the timestamp perspective. But, when they consider the different perspectives, it is simply concatenated multiple perspectives together to calculate distance, overlooking the mutual influences among different perspectives. In this paper, to address these problems, we propose a novel **Multi-view Drift Detection** method based on graph convolutional networks, named MDD. The main contributions are summarized as follows:

- Detecting the change points in logs based on multiple views, where each view is converted to an undirected graph with the k-nearest neighbor algorithm.
- Exploring the relationships between different view graphs and constructing a fusion graph through the fully connected network. Additionally, we employ the rank constraint of the Laplacian matrix to reduce the redundancy of the fusion graph.
- We utilize a two-layer GCN to capture multi-view features of log segments from two consecutive windows and use the Wasserstein distance to calculate the distance between the two log segments.
- The K-means algorithm identifies anomalies as candidate change points. Subsequently, a filter mechanism is applied to locate the actual drift points.

We evaluated our method on synthetic logs and real logs, respectively, and compared it with the baselines. The experimental results showed that our method MDD achieved the state-of-the-art level.

II. RELATED WORK

The idea of concept drift originally emerged in machine learning to address the problem of classifiers' inaccurate predictions. In the last decade, people have begun investigating concept drift issues in process mining, primarily aiming to enhance the accuracy of process discovery. Furthermore, as research has progressed, concept drift detection in process mining mainly investigates three topics: change points detection, change localization and characterization, unravel process evolution [6] [7]. In this paper, we focus on change point detection.

Detecting concept drift usually requires defining the features to be compared. These features can be obtained from event logs. In [6] [7], Bose et al. define global and local features to detect change points in the event logs. An obvious drawback of this method is that it requires users to specify the features to be extracted. If the user extracts incorrectly, the accuracy of this algorithm can be severely affected. Maaradji et al. [9] pointed out that the global and local features utilized in [6] fail to accurately capture changes in the business model. Consequently, they drew inspiration from the concurrency relationship proposed by Aalst et al. in [10] and introduced a redefined set of Run features to describe relationships between any two activities in a log. Through Run feature extraction and hypothesis testing on sub-logs in the preceding and subsequent windows, they successfully detected drift points. Later, they expanded their approach by introducing cyclic relationships and proposing Alpha features [11].

The above methods all use hypothesis testing to find drift points. However, the hypothesis testing methods typically require the reconstruction of the distribution model of features at each step of the movement. This approach can lead to inefficiency. To address this issue, Zheng et al. took an innovative approach by employing relational clustering to identify drift points, which they named Tsinghua process concept drift detection (TPCDD) [12]. However, it's important to note that TPCDD is specifically designed for offline scenarios. Moreover, Rafael et al. [13] proposed a novel clustering strategy based on activity distance to detect drift. This strategy checks each trace and can identify potential drift when creating new clusters. In [14], Seeliger et al. innovatively adopted a discovery process modeling approach. They used the heuristic Miner algorithm [15] to obtain graph metrics. However, this method only considers the relationship of edges, which leads to partial information loss and low accuracy. Marwan et al. [16] discovered an issue of inaccuracy in fixed window approaches for drift detection. Therefore, they proposed the StrProMCDD method to achieve dynamic adaptation of window size. These methods can effectively detect drift points from a single perspective of control flow, but they overlook information from other perspectives in the logs.

Literature [17] proposes a method for drift detection at the model level, trace level and activity level. The method determines whether drift has occurred by observing changes in the three metrics. However, this method requires an amount of expert knowledge. Davide et al. [18] [19] considered control flow and timestamps, defined graph distance trace (GDtrace) and graph distance time (GDtime). It utilized the DenStream clustering algorithm to detect drift points. Barbon et al. [8] integrate three perspectives of trace distance, relative time distance, and global time for clustering. Nevertheless, this method requires the users to determine the time point at which the drift occurs based on the visualization results of the real-time clustering, which inevitably imposes an extra burden on the user.

Huete et al. [20] proposed an unsupervised online concept drift detection method PrefixCDD. It stores data stream com-

ponents in a set of prefix trees and effectively uses principal component analysis (PCA) to transform discrete data into continuous data. In [21], a evaluation framework is proposed, and experimental evaluation of process concept drift techniques is conducted. Anton et al. [22] proposed a visual drift detection method (VDD) to help identify drift types by visualizing drifts and integrating formal concepts such as temporal logic and DECLARE constraints. However this method requires analysts to manually identify drifts based on correlation charts.

As mentioned above, the existing drift detection methods are usually based on a single perspective. Although some drift detection methods consider multiple perspectives, they tend to consider each perspective separately and fail to realize the fusion of each perspective for comprehensive consideration. Therefore, we propose a novel multi-views drift detection approach based on graph convolutional networks.

III. PRELIMINARIES

This section introduces some basic definitions that are used throughout the paper, with definitions borrowed from [9] [23].

[Event log] Let $\mathcal{T}$ be a finite set of activities. An event log $\mathcal{L}$ over $\mathcal{T}$ is defined as $\mathcal{L} = (\mathcal{E}, \mathcal{C}, \gamma, \tau, \succ)$, where $\mathcal{E}$ is a set of events, $\mathcal{C}$ is a set of case identifiers, $\gamma: \mathcal{E} \rightarrow \mathcal{C}$ a surjective function relating events to cases, $\tau: \mathcal{E} \rightarrow \mathcal{T}$ a function relating events to activities and $\succ \subseteq \mathcal{E} \times \mathcal{E}$ a total order on events.

[Trace] Let $\mathcal{T}$ be a finite set of activities and $\mathcal{L} = (\mathcal{E}, \mathcal{C}, \gamma, \tau, \succ)$ a log over $\mathcal{T}$. For all cases $c \in \mathcal{C}$, we define the trace $\sigma_c = \langle e_1, e_2, \dots, e_n \rangle$ as an event sequence over the event set $\mathcal{E}(\sigma_c) = \{e \in \mathcal{E} | \gamma(e) = c\}$ as all events relating to case c , where n is the number of events in $\mathcal{E}(\sigma_c)$, $e_i \in \mathcal{E}(\sigma_c)$, $i \in \{1, 2, \dots, n-1\}$ and if $1 \leq i < n$, it holds $e_i \succ e_{i+1}$. The

length of the trace σ_c is defined as the number of elements in $\mathcal{E}(\sigma_c)$, which is denoted as $|\sigma_c|$.

Definition 1 and Definition 2 elucidate the composition of an event log and define a trace as a series of chronologically ordered events. These events can pertain to activity associated with business processes and commonly encompass a timestamp, event type, resource and potentially other attributes.

[Concept drift] Let $\mathcal{P}_0, \mathcal{P}_1, \dots, \mathcal{P}_n$, be $n+1$ different process models and $t_0, t_1, \dots, t_n$ be $n+1$ moments. $\mathcal{P}(t_i) = \mathcal{P}_i$ denotes the model being used at t_i . $\mathcal{P}(t_0) = \mathcal{P}_0$ is the initial model. When moment t_i , ($0 < i \leq n$) arrives, the currently used model will change into $\mathcal{P}_i$ immediately. But the traces are still recorded in the same log. Such phenomenon is called concept drift. In this context, $t_1, \dots, t_n$ are called change points.

Definition 3 introduces the notion of concept drift, which reveals the evolution of business processes over time. At the same time, these changes are recorded in the log from different perspectives. Hence, it is meaningful to promptly find out these change points in the log, which contributes to the reliability of process mining analysis.

IV. METHODOLOGY

In this section, a novel drift detection method, named MDD, will be introduced, which is able to accurately detect change points in the log from multiple perspectives like activity, resource and transition.

Fig.2 shows the framework of MDD. First, we use two consecutive time windows to move over the event log and extract the feature vectors of each perspective of the traces within the windows. Then, the k-nearest neighbor algorithm is used to construct the knn graphs of the log fragment

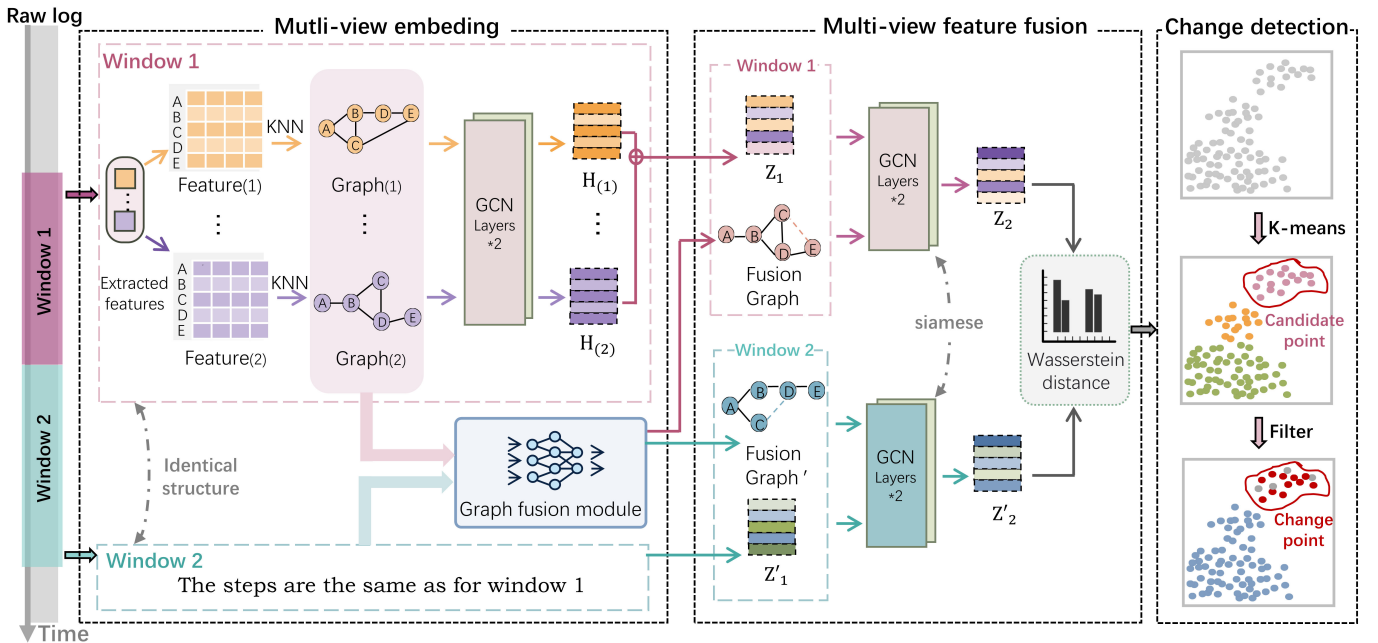


Fig. 2: Overview of our technique named MDD for multi-view drift detection

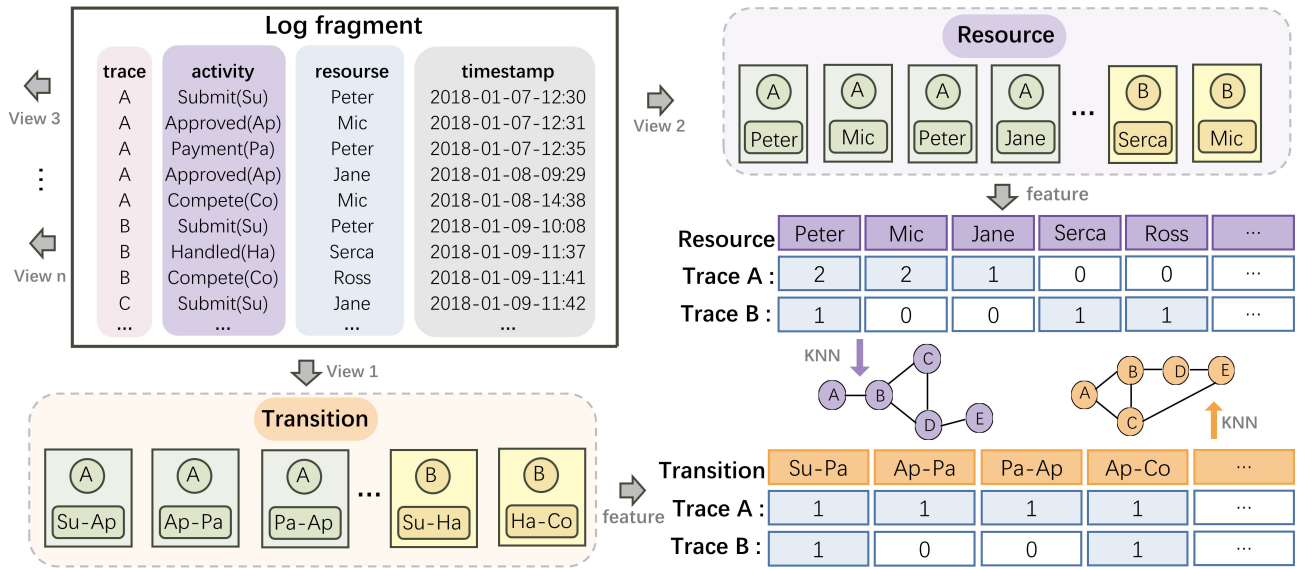


Fig. 3: Extracting multi-view features from log fragment and converting them to graphs

corresponding to each perspective within the window, and the graph convolutional networks (GCN) are used to perform the convolution operation. To obtain a more accurate and complete feature representation, we fuse the knn graphs of all perspectives into a fusion graph through the graph fusion module. Second, we use GCN again to perform convolutional operations on the fusion graph, obtaining the common representation. The distance between two windows (i.e., two common representations) is computed with the Wasserstein distance. Finally, the k-means algorithm is utilized to obtain candidate points and the actual drift point is determined by a filter mechanism.

A. Multi-view embedding

In this subsection, we will provide a detailed explanation of the following: 1) constructing the k-nearest neighbor graphs from the two log fragments, 2) embedding the graphs from each perspective, 3) constructing a fusion graph utilizing the graph fusion module.

1) *K-nearest neighbor graph construction*: We use two consecutive windows to obtain two log fragments, and then extract multi-perspective features for each log fragment. We believe that if there are differences in these features between two log fragments, then the intersection point of the two windows is a drift point. In [24], Song et al. introduced a method to describe traces from different perspectives. Inspired by it, we also extract information from traces about *activity perspective*, *resource perspective* and *transition perspective*:

–*Activity perspective*. By analyzing the activity perspective, we are able to get a more complete understanding of the distribution and frequency of activities in the trace. This approach defines features by counting the number of occurrences of specific activity in the trace. In Fig.3, we extracted a total of five unique activities from a log fragment, thus the length of the feature vector for each trace is five, where each value

represents the number of times the corresponding activity occurs in the trace.

–*Resource perspective*. The resource perspective is defined from organizational perspective, where the feature is calculated by counting the number triggered by the specified resource. As shown in Fig.3, if the resource "Peter" appears twice in trace A and the resource "Jane" appears once, the feature vector of trace A will be $\langle 2, 2, 1, 0, 0, \dots \rangle$, where each value corresponds to the number of times the corresponding resource appears in the trace.

–*Transition perspective*. The transition perspective is defined from control flow perspectives and reflects the directly follow relation between activities. For example, an activity "Su" in the log fragment is immediately followed by an activity "Ap", the feature that can be defined as "Su→Ap". The transition feature is defined by counting the number of occurrences of the transition in the trace. As shown in Fig.3, since the transitions "Su→Pa" and "Ap→Co" each occur once in Trace B, the feature vector of Trace B in the transition perspective is $\langle 1, 0, 0, 1, \dots \rangle$.

After obtaining the feature vectors from these perspectives, we construct the knn graphs using the k-nearest neighbor algorithm. For each knn graph, the nodes represent the traces in the log fragment, and the edge between nodes indicate the similarity between the traces. As we know different values of k will result in varying graph structures. In MDD, we default $k = 3$. Because if the value of k is too small, it won't accurately capture the similarity between traces. Conversely, excessive values of k will lead to redundant edges.

2) *Knn graphs embedding*: After knn graphs construction, we apply Graph Convolutional Network (abbr., GCN) [25] to capture the features from each knn graph. Due to GCN's efficient propagation mechanism, we can capture the global features of each knn graph. Its propagation rule is as follows:

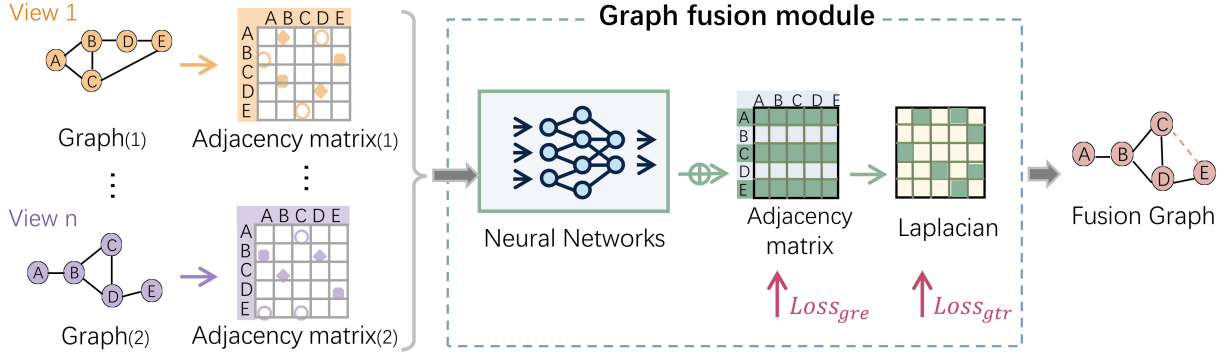


Fig. 4: The process for obtaining a fusion graph

$$H^{(l)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l-1)} W^{(l)} \right). \quad (1)$$

Where $H^{(l)} \in \mathbb{Z}^{N \times D}$ represents the convolutional result in the l^{th} layer. $\tilde{A}$ is the adjacency matrix of the graph with self-connections added. $\tilde{D}$ is the degree matrix and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. $W^{(l)}$ is the weight matrix of the l^{th} layer. $\sigma(\cdot)$ is a non-linear activation function.

It is worth noting that when performing convolution operations, the weight matrix $W^{(l)}$ is randomly generated, which may have a serious impact on the final distance calculation. Inspired by [26], we utilize **siamese mechanism** to bind the weights in GCN, as follows:

$$\begin{cases} M & \equiv N \\ m & \equiv n \\ w_i^M & \equiv w_i^N, \forall i \in [1, 2, \dots, m] \end{cases}$$

where, M and N correspond to two GCN models. $W^M = \langle w_1^M, w_2^M, \dots, w_n^M \rangle$ and $W^N = \langle w_1^N, w_2^N, \dots, w_n^N \rangle$ can respectively represent the weights of the M and N models.

The number of layers in graph convolutional networks is a hyperparameter, commonly ranging from 1 to 3 [27]. If this number is too large, it can lead to overfitting and an inability to accurately capture small differences between graphs. Conversely, if the value is too small, it may struggle to capture the global features of knn graphs. In this paper, we set this hyperparameter to 2. Specifically, the first layer consists of v view-specific GCN layers, and the inputs are the multi-view feature data $\{X_{(v)}\}_{v=1}^V$, and the corresponding knn graphs $\{A_{(v)}\}_{v=1}^V$. In the second GCN layer, the output of the first GCN layer is taken as the input, and the graph convolution operation is performed again. Eventually, we are able to learn the feature representation $\{H_{(v)}\}_{v=1}^V$ corresponding to each perspective, which is constructed as follows:

$$\hat{A}_{(v)} = \tilde{D}_{(v)}^{-\frac{1}{2}} \tilde{A}_{(v)} \tilde{D}_{(v)}^{-\frac{1}{2}}, \quad (2)$$

$$H_{(v)} = \sigma \left(\hat{A}_{(v)} \sigma \left(\hat{A}_{(v)} X_{(v)} W_{(v)}^{(1)} \right) W_{(v)}^{(2)} \right). \quad (3)$$

Where $\hat{A}_{(v)}$ is the result after symmetric normalization of the adjacency matrix. After obtaining the feature representation of each perspective, we assign a weight to each perspective to obtain the initial fusion result Z_1 . As follows:

$$Z_1 = \sum_{v=1}^V W_{(v)}^a H_{(v)}, \quad (4)$$

where $W_{(v)}^a$ is the weight matrix for each perspective.

3) *Fusion graph*: Knn graphs embedding focuses on obtaining features from different perspectives in isolation, lacking a comprehensive consideration of the structural information of log segments. Therefore, we introduce a graph fusion module to construct the overall structure of log segments.

This module utilizes fully connected neural networks to learn fusion graphs. First, we input knn graphs for each view. To achieve adaptive fusion, we add a multi-graph fusion module in the first layer of the graph fusion module, as follows:

$$G^{(1)} = \phi \left(\sum_{v=1}^V W_f \left(W_{(v)}^{g(1)} A_{(v)} + b_{(v)}^{g(1)} \right) \right), \quad (5)$$

where ϕ is the activation function of the fully connected layer. W_f denotes the attention coefficient matrix, which is used to measure the importance of different views. $W_{(v)}^{g(1)}$ and $b_{(v)}^{g(1)}$ are the weight matrix and bias of layer 1 in the graph fusion network, respectively.

The learning model of the $l - th$ layer in the graph fusion module can be represented as:

$$G^{(l)} = \phi \left(W^{g(l)} G^{(l-1)} + b^{g(l)} \right), \quad (6)$$

where $W^{g(l)}$ and $b^{g(l)}$ are the weight matrix and bias of layer l in the graph fusion network.

As shown in Fig. 4, in order to maximize the preservation of the graph information from each view, we use $loss_{gre}$ (i.e., fusion graph reconstruction loss function) to measure the degree of difference between the fusion graph and the knn graph from each view. In addition, we also use the rank constraint on the Laplace matrix based on $loss_{gtr}$ (i.e.,

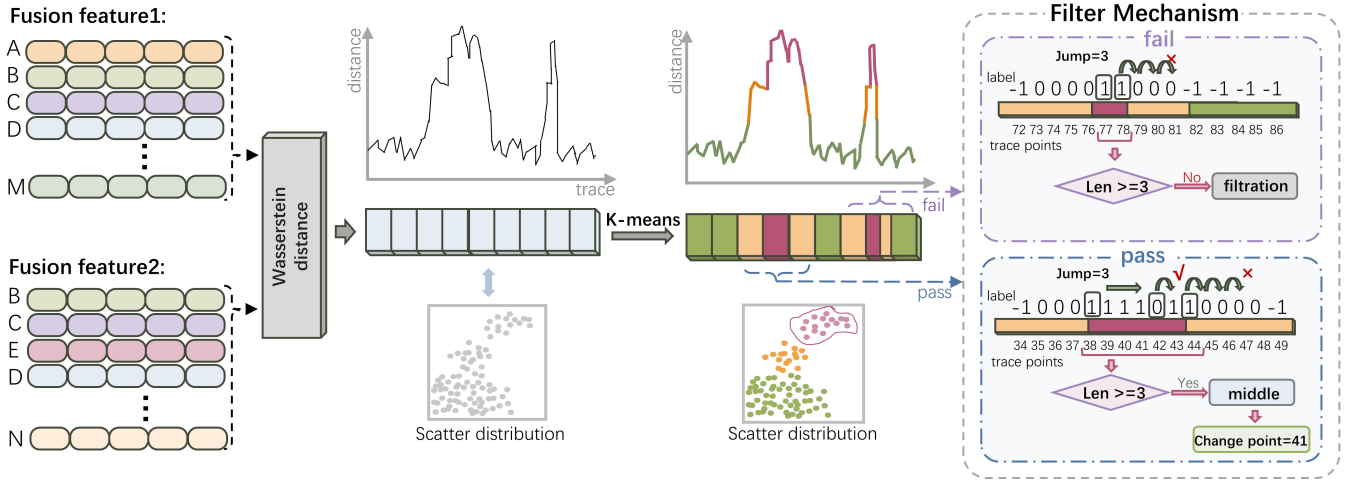


Fig. 5: An example for illustrating the detail of change detection

fusion graph trace loss function), which can help to remove redundant edges in fusion graph. As follow:

$$Loss_{gre} = \sum_{v=1}^V loss(A_{(v)}, A), \quad (7)$$

$$Loss_{gtr} = tr(Q^T L_A Q), s.t. Q^T Q = I, \quad (8)$$

$$Loss_G = Loss_{gre} + Loss_{gtr}, \quad (9)$$

where A is the adjacency matrix corresponding to the fusion graph, L_A is the Laplacian matrix of A . Q is the eigenvector matrix of A , and Q^T is the transpose matrix for Q . $tr()$ denotes the trace operator, which is used to compute the trace of the matrix.

B. Multi-view feature fusion

With multi-view graph embedding, we can obtain two sets of initial fusion result (i.e., Z_1, Z'_1) and the fusion graphs (i.e., *fusion graph*, *fusion graph'*), which correspond to two consecutive windows respectively. To improve the initial fusion result, we use GCN again to perform a two-layer convolution operation on the initial fusion result and fusion graph. In this way, we can obtain two common representations for the two windows (i.e., common representation Z_2 for window 1 and common representation Z'_2 for window 2).

The common representation is a list containing the fusion features of all the traces in the window. In other words, as long as we can calculate the distance between two common representations, we can determine whether there has been a drift between two log fragments based on this distance. In this setting, we can **Wasserstein Distance** to compute the distance between the two common representations. As follow:

$$W(Z_2, Z'_2) = \inf_{\gamma \sim \Pi(Z_2, Z'_2)} E_{v_1, v_2 \sim \gamma} [\|v_1 - v_2\|], \quad (10)$$

where v_1 and v_2 are traces from Z_2 and Z'_2 respectively. $E_{v_1, v_2 \sim \gamma} [\|v_1 - v_2\|]$ represents the expected distance between

v_1 and v_2 . $\Pi(Z_2, Z'_2)$ is represents all combinations from Z_2 to Z'_2 . γ is each possible combinations and **inf** represents finding the minimum value among all combinations.

C. Change Detection

By sliding the two windows, we can keep calculating the distances in the event logs from multiple perspectives. we believe that the larger the distance value, the higher the probability that a drift has occurred at that trace point. As shown in Fig. 5, we map the distance values onto a two-dimensional coordinate axis, where the y-axis represents the distance values and the x-axis represents trace points in the logs. Then, we use the **K-means** Algorithm to divide the points into three different levels: high, medium, and low. Here, high-level points indicate that the difference between two neighboring log fragments is significant and are labeled as 1. The medium level indicates that the difference between two neighboring log fragments is general and are labeled as 0. The low level represents that the difference between two neighboring log fragments is minor and are labeled as -1. From the multiple perspectives, we think that only significant differences indicate the possibility of drift occurrence. Therefore, the trace points labeled as 1 are considered candidate change points.

As we know, the process model usually consists of many choice and concurrency structures. So, the execution of these structures may result in an increase in distance between two log fragments. Additionally, there may be certain some noise interference in the log, such as employee operational errors or system failures. It also can affect the distance between two log fragments. Therefore, to reduce this interference, we design a simple and efficient **Filter Mechanism** to find out the actual change points from the candidate points. The essence of the filter mechanism is to determine whether drift has truly occurred by continuously changing intervals. To achieve this, we establish two hyperparameters to filter out false change points: *jump distance* and *change length*:

—*Jump distance (abbr., j)*. Due to the mutual constraints between multiple perspectives, it may lead to the alternating occurrence of label 1 and label 0 (concurrent structures can also cause this situation). As shown in Fig.5, assuming we set J to 3, when we find a trace point labeled as 1, we will jump forward 3 trace points. If there is a trace point labeled as 1 within these three points, we will take this trace point as the new starting point and jump forward again until no trace point labeled as 1 appears. Through this process, we can obtain two groups of trace points [38, 39, 40, 41, 42, 43, 44] and [77, 78].

—*Change length (abbr., l)*. If the duration of the trace points labeled as 1 is very short, it is highly likely that it is due to noise and other factors. So, we handle this issue with the hyperparameter l that is defaulted to 3. In this setting, the group [77, 78] in Fig.5 will be filtered out. Then, we have to confirm the actual change points in the group [38, 39, 40, 41, 42, 43, 44]. These points in the group represent traces in the log, with label 1 indicating they are in a drifting stage. Until the label 1 no longer appears, the evolution tends to be stable in logs, thus ending the drift.

Hence, we take the *middle point* to replace all the points in a group, where the *middle point* is defined by $middle\ point = group.get(\lfloor \frac{group.size-1}{2} \rfloor)$. The $group.size$ represents the length of the group, $group.get()$ represents selecting a specific point from the group. Finally, the actual change point in the group [38, 39, 40, 41, 42, 43, 44] is [41].

At last, all the drift points will be found in the event log through the k-means algorithm and filter mechanism. Then, we are able to accurately partition the event log into the sub-logs based on these drift points, which can provide support for process mining.

V. EXPERIMENTS

In this section, we evaluate the performance of MDD, which is implemented as a stand-alone Python application. All the code is publicly available¹.

A. Datasets

We conduct experiments on both synthetic datasets and real datasets respectively. These datasets were chosen to

comprehensively evaluate the performance of drift detection algorithms.

—Synthetic datasets, *prodrift*² is the most cited synthetic dataset, where each event log has 9 change points. This dataset consists of 72 artificial event logs and it applies twelve simple change patterns and six compound change patterns. Each change pattern has four different log sizes: 2,500 (abbr., 2.5k), 5,000 (abbr., 5k), 7,500 (abbr., 7.5k), and 10,000 (abbr., 10k).

—Real datasets, collected five logs related to travel reimbursement processes at Eindhoven University of Technology from 2017 to 2018. These logs include *Travel Permits*³(abbr., *TP*), *Domestic Declarations*⁴(abbr., *DD*), *International Declarations*⁵(abbr., *ID*), *Prepaid Travel Costs*⁶(abbr., *PTC*) and *Requests for Payment*⁷(abbr., *RP*). It is worth noting that each of these five logs only contains one drift point, as different reimbursement processes were adopted between 2017 and 2018.

B. Baselines and Metrics

a) **Baselines**: To the best of our knowledge, there are some approaches based on multi-view drift detection in process mining. We select several methods to evaluate, following the selection criteria: (i) The implementations of these methods are publicly accessible. (ii) If a method has many extended versions, we only choose the most outstanding one. (iii) The target for drift detection is event logs, not images, texts, and so on. Taking all these factors into consideration, two baseline methods are selected: **CDES** [19] and **EMD** [28].

b) **Metrics**: The F-score, as it balances precision and recall, is a common evaluation metric in the field of drift detection. It is calculated as follows: $F\text{-score} = \frac{(2 * precision * recall)}{(precision + recall)}$, where $precision = \frac{TP}{TP + FP}$, $recall = \frac{TP}{TP + FN}$. TP , FN , FP are defined as follows:

²https://data.4tu.nl/articles/dataset/Business_Process_Drift/12712436

³<https://data.4tu.nl/datasets/db35afac-2133-40f3-a565-2dc77a9329a3/1>

⁴<https://data.4tu.nl/datasets/6a0a26d2-82d0-4018-b1cd-89afb0e8627f/1>

⁵<https://data.4tu.nl/datasets/91fd1fa8-4df4-4b1a-9a3f-0116c412378f/1>

⁶<https://data.4tu.nl/datasets/fb84cf2d-166f-4de2-87be-62ee317077e5/1>

⁷<https://data.4tu.nl/datasets/a6f651a7-5ce0-4bc6-8be1-a7747effa1cc/1>

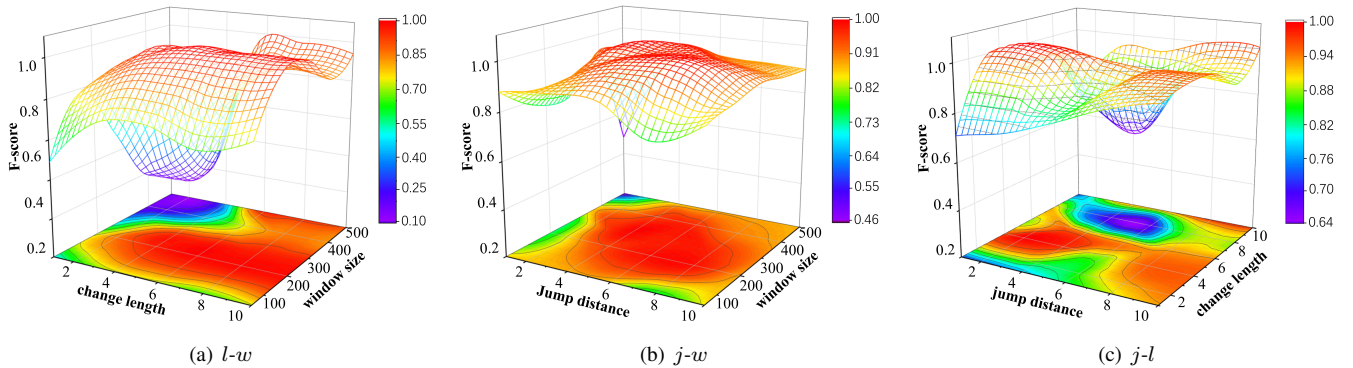


Fig. 6: The F-scores obtained under different parameter

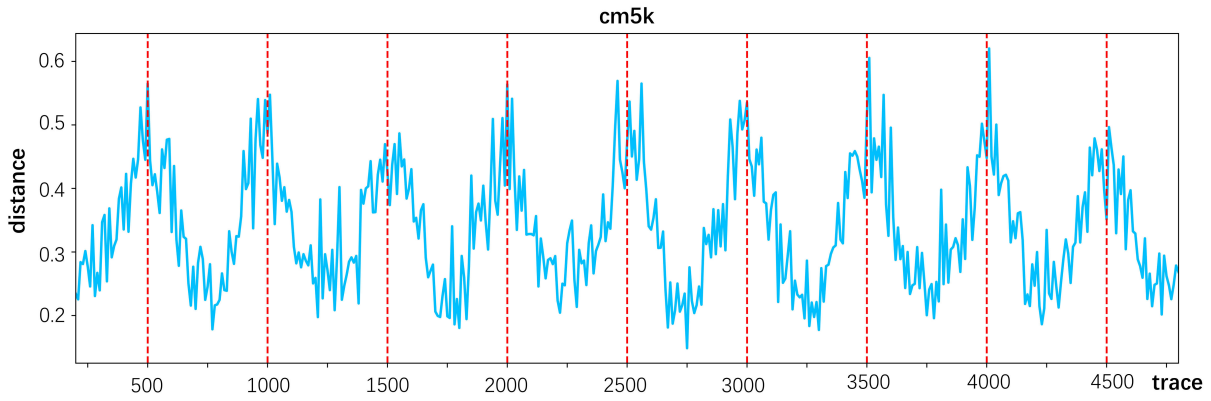


Fig. 7: Visualization of detection results in *CM5k* log from the synthetic dataset

- *TP* (True Positives): In the integer interval $[t-r, t+r]$, there is an actual drift, and the method detect a change point t .
- *FP* (False Positives): In the integer interval $[t-r, t+r]$, there is no actual drift, but the method detect a change point t .
- *FN* (False Negatives): In the integer interval $[t-r, t+r]$, there is an actual drift, but the method fails to detect a change point t .

Here, r represents the delay period, which indicates the allowable deviation range for drift detection. In this paper, the delay period is set to 50.

C. Evaluation on Different Parameter Settings

We evaluate the impact of three parameters: window size w , jump distance j and change length l . In this experiment, we set the range of w from 100 to 500, and the range of j and l from 1 to 10. To observe their mutual influence, we combine them in pairs on the *prodrift* dataset.

First, we fix $j = 3$ and observe the impact of l and w on the scores. In Fig. 6a, the red area indicates the best performance. It is observed that when $w = 200$ and $j \geq 3$, the score of MDD is the highest. Next, we fix $l = 3$ and observe the impact of j

and w . In Fig. 6b, when j and w are set to intermediate values, the scores are both good. Last, we fix $w = 200$ and varying the values of j and l . Fig. 6c demonstrates that the values of j and l have a significant impact on MDD. The highest scores are achieved when both $j = 3$ and $l = 3$.

Considering the above, we will choose the parameter combination of $w = 200$, $j = 3$ and $l = 3$ in the subsequent experiment.

D. Comparing with the Baselines

In the second experiment, we will compare the accuracy of our proposed MDD with the baselines (CDESF, EMD) on both synthetic and real datasets.

As shown in Fig. 7, we present a visualization of detection results of MDD on the *prodrift* dataset for the *CM5k* log. In this log, there are a drift every 500 trace points, and our detection is represented by the red dashed line in the figure. Our detection results align with the ground truth, demonstrating the effectiveness of the MDD.

Then, we conduct experiments on 5k logs from the *prodrift* dataset. As shown in Fig. 8, we can observe that MDD achieves perfect scores on 18 change patterns. Particularly, in the *RE* and *LP* logs, when CDESF scores are 0 and EMD

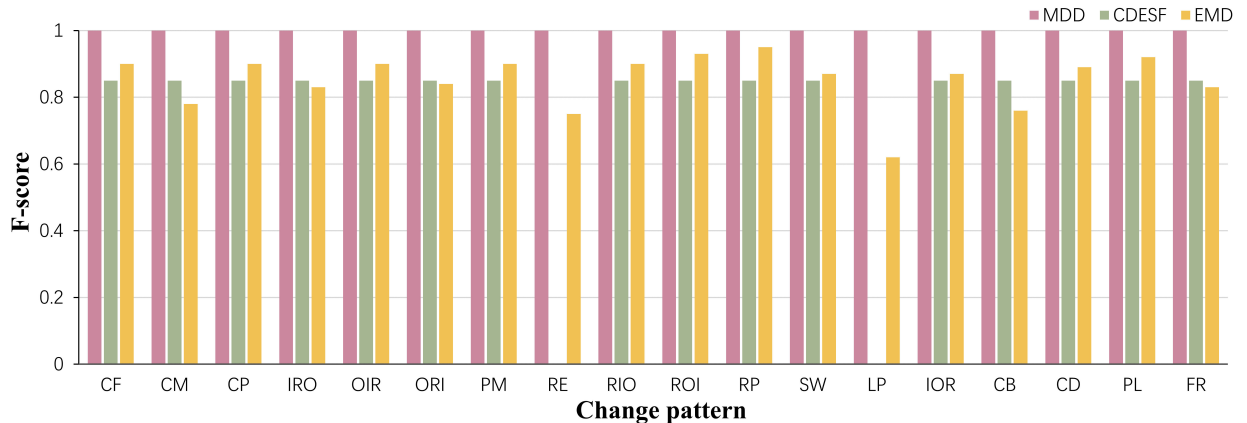


Fig. 8: Comparison of F-scores under different change patterns in the 5k logs

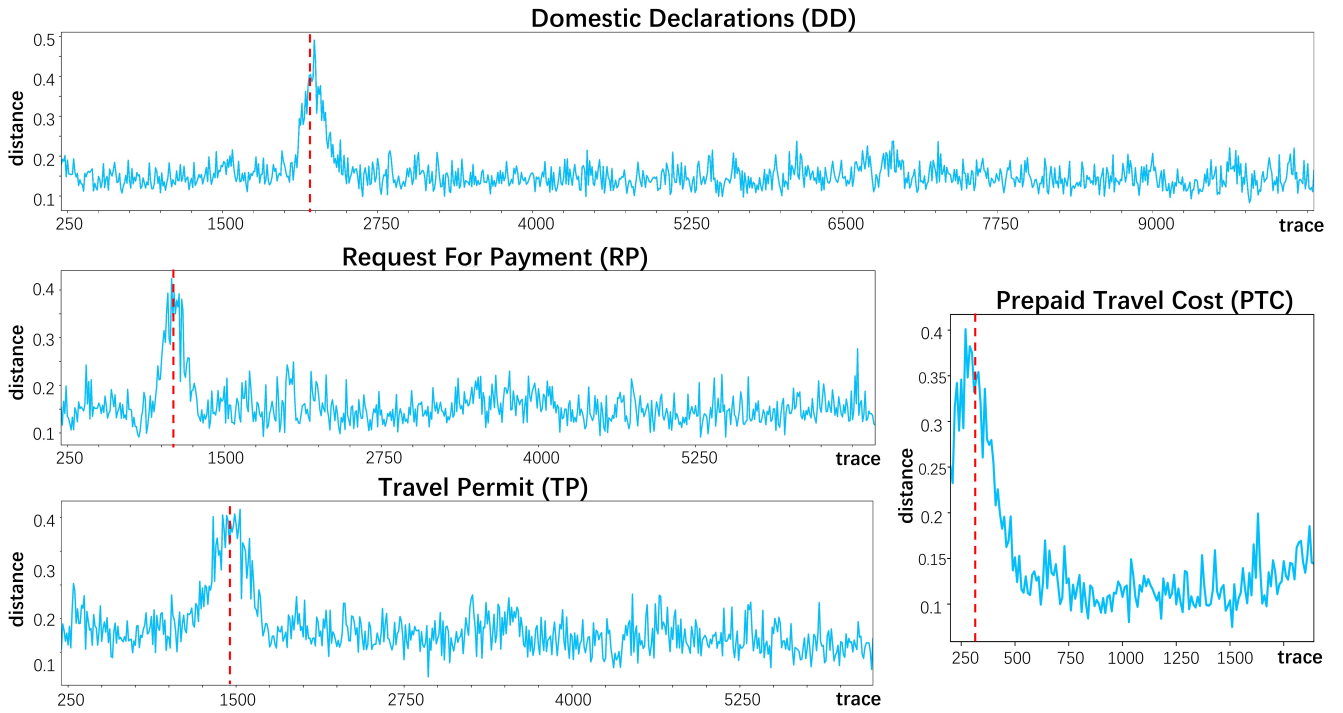


Fig. 9: Visualization of detection results in real logs

TABLE I: F-score of different methods for different logs in real logs

Method	TP	DD	ID	PTC	RP
CDESf	0.0	0.0	0.0	0.4	1.0
EMD	0.0	0.4	0.0	0.76	0.85
MDD	1.0	1.0	1.0	1.0	1.0

scores are below 0.8, MDD is still able to maintain a score of 1. Additionally, we also conduct experiments on logs of different sizes (2.5k, 7.5k, 10k), and the results show similar trends. This experiment further demonstrates the effectiveness of MDD in drift detection by fusing multiple perspectives.

Next, we will perform the comparison on the real dataset. The dataset consists of five real logs, each containing only one drift point. Due to the presence of a large number of uncommon behaviors and significant noise interference in the real dataset, the difficulty of drift detection is significantly increased. As shown in Table I, CDESf and EMD fail to find the drift points in the *Travel Permits* and *International Declarations*. Even in other logs where drift points were found, there were issues with incorrect drift points. In contrast, MDD shows excellent performance in all five logs, successfully and

accurately identifying the drift points.

As shown in Fig. 9, we present the visualizations of MDD's detection results on four real logs. The red dashed line indicates the specific positions we detected. For example, in the *DD* and *PTC* logs, the drift points we detected are the 2235th and 297th traces, respectively, while the actual drifts are the 2215th and 314th traces, respectively. According to the evaluation metric, our detected results are considered true positives. These visualizations further illustrate the robustness and accuracy of MDD when dealing with complex situations in real datasets.

E. Ablation Study

To evaluate the contributions of each component in our proposed MDD, we designed two experiments: 1) fusion graph- $\emptyset$ represents discarding the fusion graph module and directly using graph convolutional networks to fuse the features from multiple perspectives. 2) filter mechanism- $\emptyset$ removing the filter mechanism, and use the midpoints of the k-means clustering result as the drift points.

The experimental results are shown in Table II. Without the graph fusion module, the performance will decrease by about

TABLE II: Ablation analysis in different logs

Method	Synthetic datasets				Real datasets				
	2.5K	5K	7.5K	10K	TP	DD	ID	PTC	RP
fusion graph- $\emptyset$	0.91	0.95	0.94	0.92	0.91	0.87	0.86	0.81	0.93
filter mechanism- $\emptyset$	0.82	0.79	0.83	0.83	0.81	0.82	0.78	0.81	0.83
MDD	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

10%. This is because the overall structural information of the log is not taken into account, resulting in an inability to capture the relationships between the perspectives. The lack of filter mechanism will result in performance decrease of about 20%. Specifically, on synthetic datasets, not using a filter mechanism may increase the number of False Positives (*FP*) and thus reduce the overall accuracy.

VI. CONCLUSION AND FUTURE WORK

Information systems generate a vast amount of event logs stored in databases while providing services. Conducting drift detection on these logs is a highly meaningful topic. In this paper, we propose a novel multi-view drift detection method named MDD. MDD first extracts features from different perspectives within the logs and transforms them into the k-nearest neighbors graphs. Then, it utilizes graph convolutional networks to fuse features from knn graphs and form the common representation. Subsequently, the k-means algorithm and filter mechanism are used to detect change points in the logs based on the common representations. We conduct comparative experiments on synthetic and real datasets. The experimental results show that MDD outperforms the state-of-the-art baselines.

In the future, we will expand the work content of this article to adapt to more different types of concept drift in process mining, such as gradual drift and recurring drift.

ACKNOWLEDGMENT

The work was supported by the general project numbered KM202310028003 of Beijing Municipal Education Commission.

REFERENCES

- [1] W. M. Van der Aalst, "Using process mining to bridge the gap between bi and bpm," *Computer*, vol. 44, no. 12, pp. 77–80, 2011.
- [2] S. J. Leemans, D. Fahland, and W. M. Van Der Aalst, "Discovering block-structured process models from event logs containing infrequent behaviour," in *Business Process Management Workshops: BPM 2013 International Workshops, Beijing, China, August 26, 2013, Revised Papers 11*. Springer, 2014, pp. 66–78.
- [3] Q. Guo, L. Wen, J. Wang, Z. Yan, and P. S. Yu, "Mining invisible tasks in non-free-choice constructs," in *International Conference on Business Process Management*. Springer, 2015, pp. 109–125.
- [4] S. J. Leemans, A. F. Syring, and W. M. van der Aalst, "Earth movers' stochastic conformance checking," in *Business Process Management Forum: BPM Forum 2019, Vienna, Austria, September 1–6, 2019, Proceedings 17*. Springer, 2019, pp. 127–143.
- [5] D. Fahland and W. M. van Der Aalst, "Model repair—aligning process models to reality," *Information Systems*, vol. 47, pp. 220–243, 2015.
- [6] R. J. C. Bose, W. M. van der Aalst, I. Žliobaitė, and M. Pechenizkiy, "Handling concept drift in process mining," in *Advanced Information Systems Engineering: 23rd International Conference, CAiSE 2011, London, UK, June 20–24, 2011. Proceedings 23*. Springer, 2011, pp. 391–405.
- [7] R. J. C. Bose, W. M. Van Der Aalst, I. Žliobaitė, and M. Pechenizkiy, "Dealing with concept drifts in process mining," *IEEE transactions on neural networks and learning systems*, vol. 25, no. 1, pp. 154–171, 2013.
- [8] S. Barbon Junior, G. M. Tavares, V. G. T. da Costa, P. Ceravolo, and E. Damiani, "A framework for human-in-the-loop monitoring of concept-drift detection in event log stream," in *Companion proceedings of the the web conference 2018*, 2018, pp. 319–326.
- [9] A. Maaradji, M. Dumas, M. La Rosa, and A. Ostovar, "Fast and accurate business process drift detection," in *Business Process Management: 13th International Conference, BPM 2015, Innsbruck, Austria, August 31–September 3, 2015, Proceedings 13*. Springer, 2015, pp. 406–422.
- [10] W. Van der Aalst, T. Weijters, and L. Maruster, "Workflow mining: Discovering process models from event logs," *IEEE transactions on knowledge and data engineering*, vol. 16, no. 9, pp. 1128–1142, 2004.
- [11] A. Ostovar, A. Maaradji, M. La Rosa, A. H. ter Hofstede, and B. F. van Dongen, "Detecting drift from event streams of unpredictable business processes," in *Conceptual Modeling: 35th International Conference, ER 2016, Gifu, Japan, November 14–17, 2016, Proceedings 35*. Springer, 2016, pp. 330–346.
- [12] C. Zheng, L. Wen, and J. Wang, "Detecting process concept drifts from event logs," in *On the Move to Meaningful Internet Systems. OTM 2017 Conferences: Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23–27, 2017, Proceedings, Part I*. Springer, 2017, pp. 524–542.
- [13] R. Accorsi and T. Stocker, "Discovering workflow changes with time-based trace clustering," in *International Symposium on Data-Driven Process Discovery and Analysis*. Springer, 2011, pp. 154–168.
- [14] A. Seeliger, T. Nolle, and M. Mühlhäuser, "Detecting concept drift in processes using graph metrics on process graphs," in *Proceedings of the 9th Conference on Subject-Oriented Business Process Management*, 2017, pp. 1–10.
- [15] A. J. Weijters, W. M. van Der Aalst, and A. A. De Medeiros, "Process mining with the heuristicsminer algorithm," 2006.
- [16] M. Hassani, "Concept drift detection of event streams using an adaptive window," in *33rd International ECMS Conference on Modelling and Simulation, ECMS 2019*, 2019, pp. 230–239.
- [17] A. P. Kurniati, C. McInerney, K. Zucker, G. Hall, D. Hogg, and O. Johnson, "A multi-level approach for identifying process change in cancer pathways," in *International Conference on Business Process Management*. Springer, 2019, pp. 595–607.
- [18] G. M. Tavares, P. Ceravolo, V. G. T. Da Costa, E. Damiani, and S. B. Junior, "Overlapping analytic stages in online process mining," in *2019 IEEE International Conference on Services Computing (SCC)*. IEEE, 2019, pp. 167–175.
- [19] D. Mora, P. Ceravolo, E. Damiani, G. Marques Tavares *et al.*, "The cdesf toolkit: An introduction," in *CEUR WORKSHOP PROCEEDINGS*, vol. 2703. CEUR-WS, 2020, pp. 47–50.
- [20] J. Huete, A. A. Qahtan, and M. Hassani, "Prefixcdd: Effective online concept drift detection over event streams using prefix trees," in *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2023, pp. 328–333.
- [21] J. N. Adams, C. Pitsch, T. Brockhoff, and W. M. P. van der Aalst, "An experimental evaluation of process concept drift detection," *Proc. VLDB Endow.*, vol. 16, no. 8, p. 1856–1869, apr 2023. [Online]. Available: <https://doi.org/10.14778/3594512.3594517>
- [22] A. Yeshchenko, C. Di Ciccio, J. Mendling, and A. Polyvyanyy, "Visual drift detection for event sequence data of business processes," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 8, pp. 3050–3068, 2021.
- [23] L. Lin, L. Wen, L. Lin, J. Pei, and H. Yang, "Lcdd: detecting business process drifts based on local completeness," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2086–2099, 2020.
- [24] M. Song, C. W. Günther, and W. M. Van der Aalst, "Trace clustering in process mining," in *Business Process Management Workshops: BPM 2008 International Workshops, Milano, Italy, September 1–4, 2008. Revised Papers 6*. Springer, 2009, pp. 109–120.
- [25] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [26] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: online learning of social representations," *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2014.
- [27] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 7370–7377.
- [28] T. Brockhoff, M. S. Uysal, and W. M. van der Aalst, "Time-aware concept drift detection using the earth mover's distance," in *2020 2nd International Conference on Process Mining (ICPM)*. IEEE, 2020, pp. 33–40.